

Parsing

CMSC 323: Design and Implementation of Programming Languages

This lecture note is adapted from content created by Robby Findler (Northwestern University) and Vincent St-Amour (Northwestern University) and Shriram Krishnamurthi in *Programming Languages: Application and Interpretation*.

Parsing

It is tedious to write programs using the AST, so we use simpler notation to write our programs. We will call this our **input or surface syntax** (I sometimes use the term **concrete syntax**)

Converting the input syntax into AST is called **parsing**

We pick a syntax in our class that resembles the parenthetical racket syntax: we call the syntax an **s-expression**.

Parsing

Again, a parser maps language
syntax to the AST

```
{with {x 10}
  {- 20 {+ {+ x x}
           {+ x x}}}}
```

vs

```
(with 'x (num 10)
  (sub (num 20) (plus (plus (var 'x) (var 'x))
                       (plus (var 'x) (var 'x))))))
```

S-Expressions

A back-tick followed by a Racket term is of type S-Expression

Examples:

``1`

``2.3`

``-40`

``{+ 1 2}`

``{+ {+ 2 3}}`

In Racket (and PLAIT), the back-tick marks what follows as having type S-Exp. Note that this is not a string, it is a symbol. There is no notion of individual characters of an S-Exp. (i.e., It is atomic.)

← This is one symbol, not 11 characters.

S-Expression Type

Determining the S-Exp type:

`(s-exp-number? `1)`  $\Rightarrow$  `#t`

`(s-exp-list? `1)`  $\Rightarrow$  `#f`

`(s-exp-number? `{+ 1 2})`  $\Rightarrow$  `#f`

`(s-exp-list? `{+ 1 2})`  $\Rightarrow$  `#t`

An s-exp-list must be contained in `()` or `{}` and everything in the parentheses must be an s-exp.

Ex. ``{lambda {x} 10}`

Extracting the S-Expression ...

Extracting the data enclosed in the s-exp type

```
(s-exp->number `1) ;=> 1
```

```
(s-exp->list `{+ 1 2}) ;=> (list `+ `1 `2)
```

Symbol S-Expression

Symbol S-Expressions are like strings but different in operations and performance. Whereas there are numerous string operations, symbols are treated **atomically**; other than being converted to strings, the only other operation they support is equality. But in return, symbols can be checked for equality in constant time.

They have the same syntax as Racket variables.

S-Expression Symbol

(s-exp-symbol? `+) ; => #t

(s-exp->symbol `+) ; => '+'

Writing a Parser

```
(test (parse `1) (num 1))  
(test (parse `z) (var 'z))  
(test (parse `{+ 1 2}) (plus (num 1) (num  
2)))  
. . .
```

Exercise

- ◆ Complete the with block of the parse function.
- ◆ Include tests for your with
- ◆ Include a case in your case form that takes care of an unexpected symbol as the first item in your list.
- ◆ Now write test cases that parse and interpret an expression in our surface syntax